

HALF ADDER

The screenshot shows the VS Code interface with the Explorer panel on the left and the Editor panel on the right. The Explorer panel shows a project structure with files like `xor_gate.v`, `half_adder.v`, and `half_adder_tb.v`. The Editor panel displays the Verilog code for `half_adder.v`, which is using Behavioral Modeling. The code defines a module `half_adder` with inputs `a` and `b`, and outputs `sum` and `carry`. It uses an `always` block to implement the logic.

```
1 //Behavioural Modeling
2 module half_adder(
3     input a,
4     input b,
5     output reg sum,
6     output reg carry
7 );
8
9     always @(*)
10    begin
11        if(a==0 && b==0)begin
12            sum=0; carry=0;
13        end
14        else if(a==0 && b==1)begin
15            sum=1; carry=0;
16        end
17        else if(a==1 && b==0)begin
18            sum=1; carry=0;
19        end
20        else if(a==1 && b==1)begin
21            sum=0; carry=1;
22        end
23    end
24 endmodule
25
```

BEHAVIOURAL MODELING

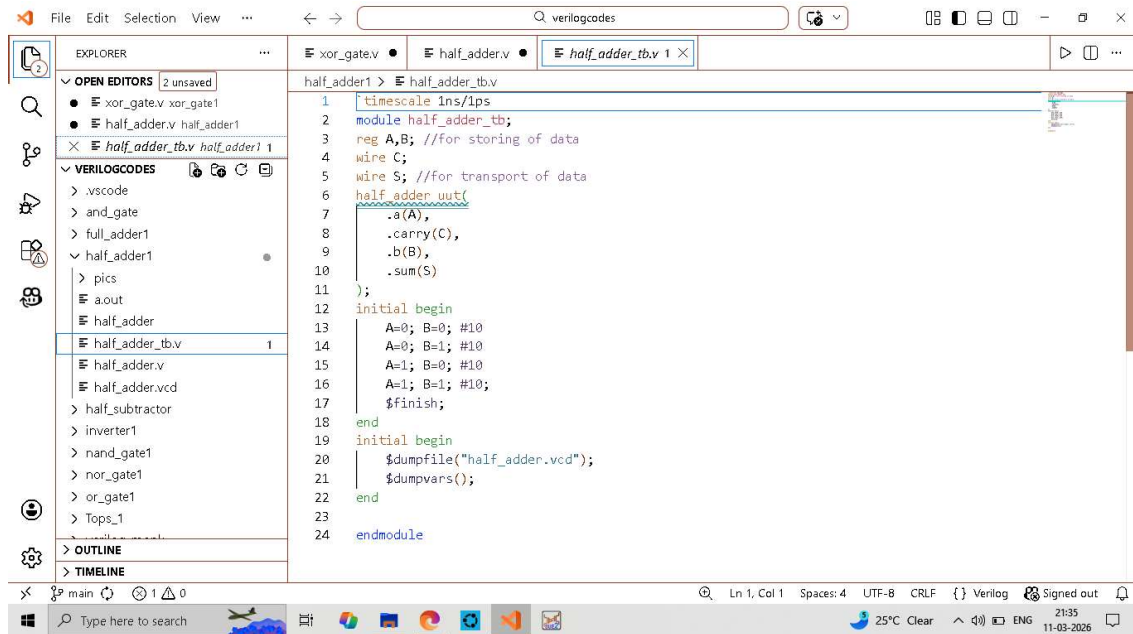
The screenshot shows the VS Code interface with the Explorer panel on the left and the Editor panel on the right. The Explorer panel shows a project structure with files like `xor_gate.v`, `half_adder.v`, and `half_adder_tb.v`. The Editor panel displays the Verilog code for `half_adder.v`, which is using Behavioral Modeling. The code defines a module `half_adder` with inputs `a` and `b`, and outputs `sum` and `carry`. It uses an `assign` statement to implement the logic.

```
26
27 //Data Flow Modeling
28 module half_adder(
29     input a,
30     input b,
31     output sum,
32     output carry
33 );
34
35     assign sum = a^b;
36     assign carry = a&b;
37 endmodule
38
39 //Gate Level Modeling
40 module half_adder
41 (
42     input a,b,
43     output sum,carry
44 );
45
46     xor(sum,a,b);
47     and(carry,a,b);
48 endmodule
49
50
```

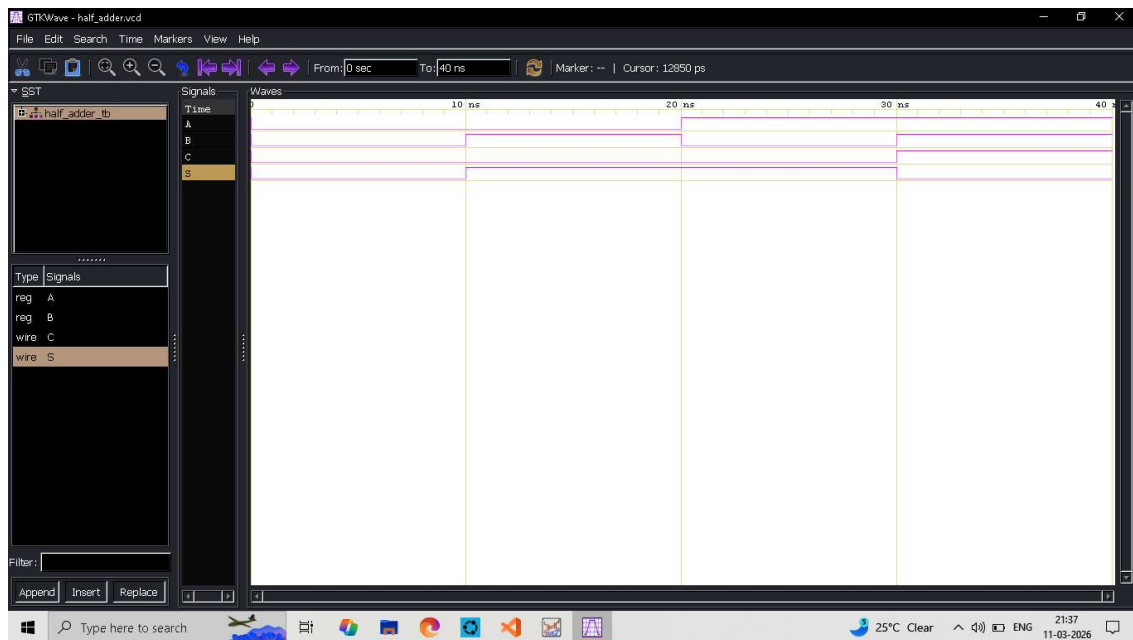
GATE LEVEL MODELING

&

DATAFLOW MODELING



TESTBENCH



WAVEFORM

HALF SUBTRACTOR

The screenshot shows the Verilog code for a half subtractor using behavioural modeling. The code is written in a text editor with a sidebar showing the project structure. The code defines a module `half_sub` with inputs `a` and `b`, and outputs `diff` and `borrow`. It uses an `always` block to model the subtraction logic.

```
1 //Behavioural Modeling
2 module half_sub(
3     input a,
4     input b,
5     output reg diff,
6     output reg borrow
7 );
8     always @(a,b)
9     begin
10         if(a==0 && b==0)begin
11             diff=0; borrow=0;
12         end
13         else if(a==0 && b==1)begin
14             diff=1; borrow=1;
15         end
16         else if(a==1 && b==0)begin
17             diff=1; borrow=0;
18         end
19         else if(a==1 && b==1)begin
20             diff=0; borrow=1;
21         end
22     end
23 endmodule
24
25
26
```

BEHAVIOURAL MODELING

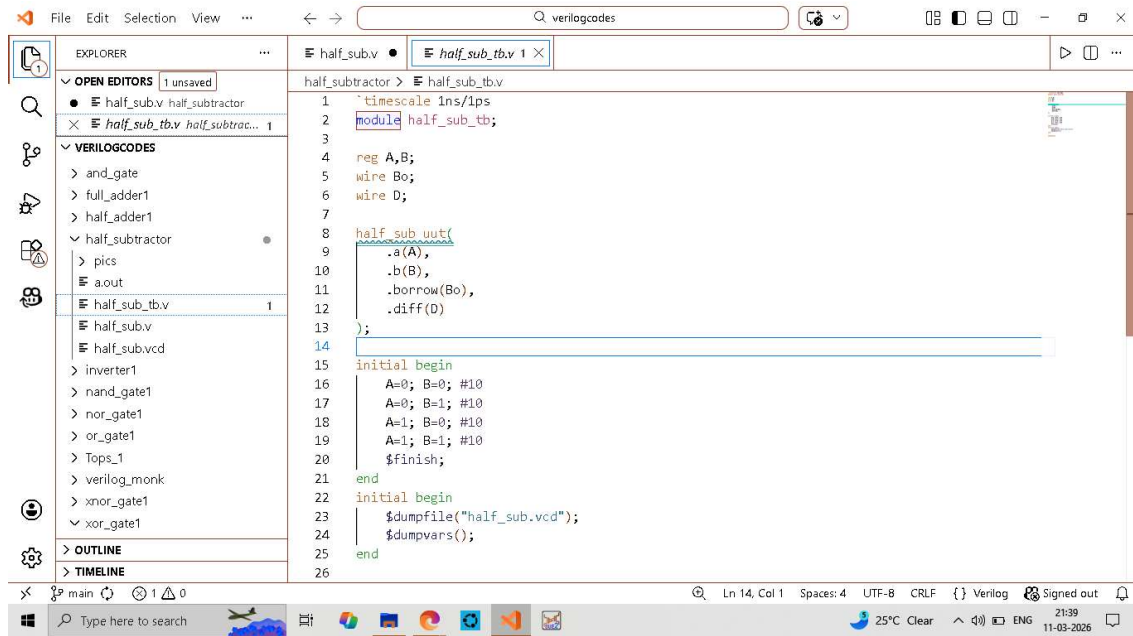
The screenshot shows the Verilog code for a half subtractor using behavioural modeling. The code is written in a text editor with a sidebar showing the project structure. The code defines a module `half_sub` with inputs `a` and `b`, and outputs `diff` and `borrow`. It uses an `assign` statement to model the subtraction logic.

```
26 //Data Flow Modeling
27 module half_sub(
28     input a,
29     input b,
30     output diff,
31     output borrow
32 );
33
34
35     assign diff = a^b;
36     assign borrow = (~a)&b;
37 endmodule
38
39 //Gate Level Modeling
40 module half_sub
41 (
42     input a,b,
43     output diff,borrow
44 );
45
46     wire a_reg;
47     xor(diff,a,b);
48     not(a_reg,a);
49     and(borrow,a_reg,b);
50 endmodule
```

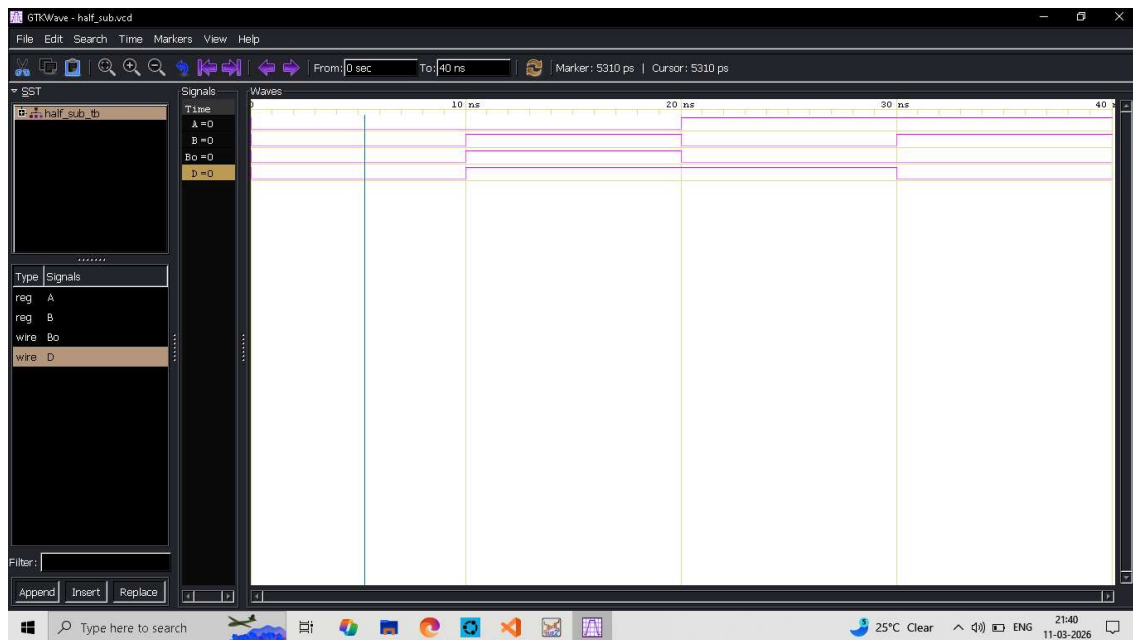
GATELEVEL MODELING

&

DATAFLOW MODELING



TESTBENCH



WAVEFORM